BERGISCHE UNIVERSITÄT WUPPERTAL

# Final Assignment Lab-Course

Wellinton C. Scheffer

October 20, 2026

## 1 The Outer Solar System

> The evolution of the entire planetary system has been numerically integrated for a time span of nearly 100 million years. This calculation confirms that the evolution of the solar system as a whole is chaotic. (G.J. Sussman & J. Wisdom, 1992)

We next apply our methods to the system which describes the motion of the five outer planets relative to the sun. This system has been studied extensively by astronomers. The problem is a Hamiltonian system

$$\dot{p} = -H_q(p,q), \qquad \dot{q} = H_p(p,q) \tag{1.1}$$

(N -body problem) with

$$H(p,q) = \frac{1}{2}\sum_{i=0}^{5}\frac{1}{m_i}p_i^t p_i - G\sum_{i=1}^{5}\sum_{j=0}^{i-1}\frac{m_i m_j}{||q_i - q_j||} \tag{1.2}$$

Here $p$ and $q$ are the vectors composed by the vectors $pi, qi \in \mathbb{R}3$ (momenta and positions), respectively. The chosen units are: masses relative to the sun, so that the sun has mass 1. We have taken

$$m_0 = 1.00000597682 \tag{1.3}$$

to take account of the inner planets. Distances are in astronomical units ($1[A,U] = 149.597.870[km]$), times in earth days, and the gravitational constant is

$$G = 2.95912208286 \cdot 10^{-4} \tag{1.4}$$

The initial values for the sun are taken as $q_0(0) = (0,0,0)^T$ and $\dot{q_0} = (0,0,0)^T$. All others data (masses of the planets and the initial positions and initial velocities) are given in Table 1.1. The initial data is taken from "Ahnerts Kalende für Sternfreunde 1994", Johann Ambrosius Barth Verlag 1993, and they correspond September 5, 1994 at 0h00.

Table 1.1: Data for the outer solar system

| planet | mass | initial position | initial velocity |
|---|---|---|---|
| Jupiter | $m_1 = 0.000954786104043$ | −3.5023653 | 0.00565429 |
| | | −3.8169847 | −0.00412490 |
| | | −1.5507963 | −0.00190589 |
| Saturn | $m_2 = 0.000285583733151$ | 9.0755314 | 0.00168318 |
| | | −3.0458353 | 0.00483525 |
| | | −1.6483708 | 0.00192462 |
| Uranus | $m_3 = 0.0000437273164546$ | 8.3101420 | 0.00354178 |
| | | −16.2901086 | 0.00137102 |
| | | −7.2521278 | 0.00055029 |
| Neptune | $m_4 = 0.0000517759138449$ | 11.4707666 | 0.00288930 |
| | | −25.7294829 | 0.00114527 |
| | | −10.8169456 | 0.00039677 |
| Pluto | $m_5 = \frac{1}{1.3 \cdot 10^8}$ | −15.5387357 | 0.00276725 |
| | | −25.2225594 | −0.00170702 |
| | | −3.1902382 | −0.00136504 |

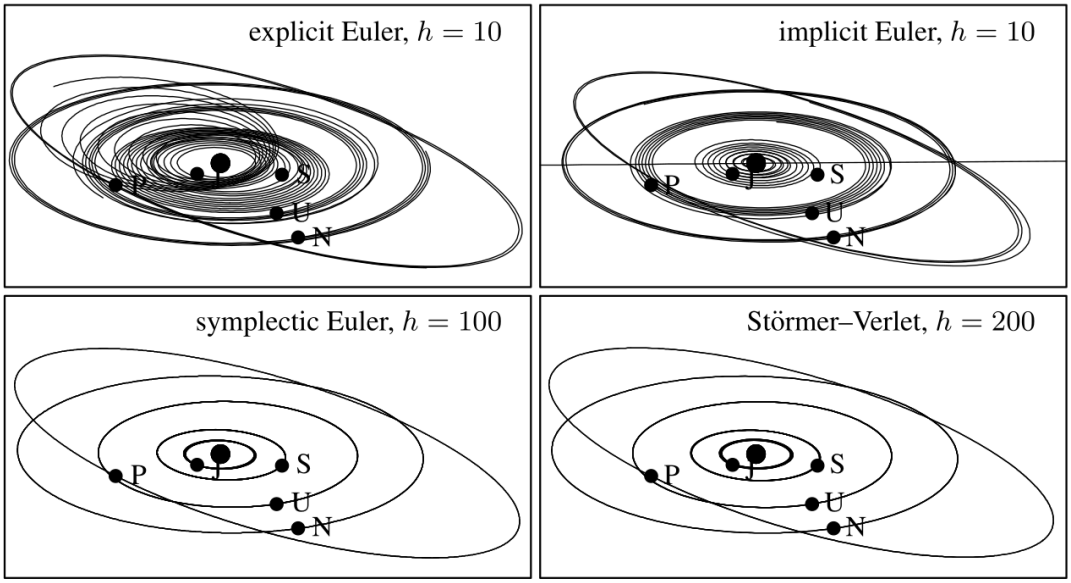



Figure 1.1: Solutions of the outer-linear systems

To this system, we apply the explicit and implicit Euler methods with step ize $h = 10$, the symplectic Euler and the Störmer- Verlet method with much larger step sizes $h = 100$ and $h = 200$, respectively, all over a time periods of 200 000 days. The numerical solution (see Fig. 1.1) behaves similarly to that for the Kepler problem. With the explicit Euler method, the planets have increasing energy, they spiral outwards, Jupiter approaches Saturn which leaves the plane of the two-body motion. With the implicit Euler method, the planets (fist Jupiter and then Saturn) fall into the sun and are thrown far away. Both the symplectic Euler method and the Störmer-Verlet scheme show the correct behaviour. An integration over a much longer time of say several million years does not deteriorate this behaviour. Let us remark that Sussman & Wisdom (1992) have integrated the outer solar system with special geometric integrators.

## 2 TASK 1

In order to reproduce the plot from the books, was used the explicit Euler method,

$$\mathbf{p}_{n+1} = \mathbf{p}_n + h\dot{\mathbf{p}}_n = \mathbf{p}_n + h\left(-\frac{\partial H}{\partial \mathbf{q}}\Big|_n\right), \tag{2.1}$$

$$\mathbf{q}_{n+1} = \mathbf{q}_n + h\dot{\mathbf{q}}_n = \mathbf{q}_n + h\left(\frac{\partial H}{\partial \mathbf{p}}\Big|_n\right). \tag{2.2}$$

which is a first-order explicit method, it evaluates derivatives at the current time step, and is simple but not symplectic, leading to artificial energy increase and instability over long

simulations. Also the implicit Euler methods,

$$\mathbf{p}_{n+1} = \mathbf{p}_n + h\left(-\frac{\partial H}{\partial \mathbf{q}}\Big|_{n+1}\right), \tag{2.3}$$

$$\mathbf{q}_{n+1} = \mathbf{q}_n + h\left(\frac{\partial H}{\partial \mathbf{p}}\Big|_{n+1}\right). \tag{2.4}$$

is a first-order implicit method, whose derivatives are evaluated at the future time step, requiring the solution of a non-linear system (via Newton's method), it is more stable than explicit Euler but dissipative (energy decreases artificially) and not symplectic. The Symplectic Euler,

$$\mathbf{p}_{n+1} = \mathbf{p}_n + h\left(-\frac{\partial H}{\partial \mathbf{q}}\Big|_{n}\right), \tag{2.5}$$

$$\mathbf{q}_{n+1} = \mathbf{q}_n + h\left(\frac{\partial H}{\partial \mathbf{p}}\Big|_{n+1}\right). \tag{2.6}$$

First-order symplectic method, it updates momentum explicitly using current positions and then updates positions using the new momentum, being symplectic, it conserves energy much better over long times and allows larger time steps. And, lastly the Störmer-Verlet,

$$\mathbf{p}_{n+1/2} = \mathbf{p}_n + \frac{h}{2}\left(-\frac{\partial H}{\partial \mathbf{q}}\Big|_{n}\right), \tag{2.7}$$

$$\mathbf{q}_{n+1} = \mathbf{q}_n + h\left(\frac{\partial H}{\partial \mathbf{p}}\Big|_{n+1/2}\right), \tag{2.8}$$

$$\mathbf{p}_{n+1} = \mathbf{p}_{n+1/2} + \frac{h}{2}\left(-\frac{\partial H}{\partial \mathbf{q}}\Big|_{n+1}\right). \tag{2.9}$$

Second-order symplectic integrator (also known as leapfrog or Verlet velocity form), it splits the time step symmetrically, providing excellent long-term energy conservation and stability, making it ideal for Hamiltonian systems like N-body gravitational problems.

Listing 1: Code used for Newton iteration

```
import numpy as np
from scipy.linalg import lu_factor, lu_solve

def APPROX_JACOBIAN(F, x):
    n = len(x) # n receive the lenght of x if x = [x1, x2, x3] n =
        3
    J = np.zeros((n, n)) # create a jacobian matrix full of zeros,
        n lines andn rows
    eps0 = 1e-16 # machine precision
    delta = np.sqrt(eps0) * np.maximum(1.0, np.abs(x)) #the sqrt
       of machine precision is the step,
    # which if we use it large, truncation erro domains, small the
        rownsdorff error domain, so,
```

```
    # the best step is the balance between them which is the sqrt
       of machine precision times the
    # maximum value (1.0 if x is small, and x iF x is large, so,
       the step can follow the order
    # of magnetude)
    Fx = F(x)

    for j in range(n):
        x_temp = x.copy() # copy of x
        x_temp[j] = x_temp[j] + delta[j]
        Fj = F(x_temp)
        J[:, j] = (Fj - Fx)/delta[j]
    return J

def NEWTON(F, x0, TOL, MAX_ITER, DF=None, Simplified = 0):

    x = np.copy(x0)

    if Simplified == 1:
        J0 = DF(x0) if DF is not None else APPROX_JACOBIAN(F, x0)

        luj, pivj = lu_factor(J0)

        for k in range(MAX_ITER):
            Fx = F(x)
            delta_x = lu_solve((luj, pivj), -Fx)
            x = x + delta_x
            if np.linalg.norm(delta_x,2)<TOL:
                return x, k+1
        return x, MAX_ITER
    else:
        for k in range(MAX_ITER):
            Fx = F(x)

            if DF is not None:
                J =DF(x)
            else:
                J = APPROX_JACOBIAN(F, x)

            lu, piv = lu_factor(J)
            delta_x = lu_solve((lu, piv), -Fx)
            x = x + delta_x

            if np.linalg.norm(delta_x, 2) < TOL:
                return x, k+1
        return x, MAX_ITER
```

Listing 2: Main code to generate the plots

```
import numpy as np
```

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from Newton import NEWTON, APPROX_JACOBIAN

G = 2.95912208286 * 10**(-4) # Graviatational constant

masses = np.array([ # Mass
    1.00000597682, #m0, Sun + Inner planet
    0.000954786104043, # m1 Jupiter
    0.000285583733151, # m2 Saturn
    0.0000437273164546, #m3 Uranus
    0.0000517759138449, # m4 Neptune
    1/(1.3*10**8), # m5 Pluto
    ])

q_init = np.array([ # Positons
    [0.0, 0.0, 0.0], # Sun
    [-3.5023653, -3.8169847, -1.5507963], # Jupiter position
    [9.0755314, -3.0458353, -1.6483708], # Saturn Position
    [8.3101420, -16.2901086, -7.2521278], # Uranus position
    [11.4707666, -25.7294829, -10.8169456], # Neptune possition
    [-15.5387357, -25.2225594, -3.1902382], # Pluto position
    ])

v_init = np.array([ # Velcities
    [0.0, 0.0, 0.0], # Sun position
    [0.00565429, -0.00412490, -0.00190589], # Jupiter Velocity
    [0.00168318, 0.00483525, 0.00192462], # Saturn Velocity
    [0.00354178, 0.00137102, 0.00055029], # Uranus Velocity
    [0.00288930, 0.00114527, 0.00039677], # Neptune Velocity
    [0.00276725, -0.00170702, -0.00136504], # PLuto Velocit
    ])

p_init = masses[:, np.newaxis]*v_init # initial moment p = m*v

def Compute_Acc(q, masses, G): # calculate the \dot{p} = -\partial
    H/\partial q

    n = len(masses) # number os bodies
    dp = np.zeros_like(q) # same size as q, but fill of zeros

    for i in range(n): # double loop to calculate the iteration
       between bodiies
        for j in range(n):
            if i != j: # avoid the body to interact with itself
                r_ij = q[i] - q[j]
                dist = np.linalg.norm(r_ij) + 1e-12 # euclidian
                   distance
                dp[i] += -G * masses[i] * masses[j] * r_ij / (dist
```

```
                    **3)
    return dp

def compute_Vel(p, masses): # calculate de \dot{q} = \partial H/ \
    partial p

    return p/masses[:, np.newaxis]

def explicita_Euler(p, q, h, masses, G):

    dp = Compute_Acc(q, masses, G)
    dq = compute_Vel(p, masses)

    p_new = p+h*dp # explicit updates, pn+1 = pn + h \dot{pn}
    q_new = q+h*dq

    return p_new, q_new

def implicit_Euler(p,q,h, masses, G):

    n = len(masses) # number of bodies

    def F(x):

        p_new = x[:n*3].reshape(n,3) # extract pn+ from the first
            n*3 elements
        q_new = x[n*3:].reshape(n,3)

        dp_new = Compute_Acc(q_new, masses, G)
        dq_new = compute_Vel(p_new, masses)

        F_p = p_new - p - h*dp_new
        F_q = q_new - q - h*dq_new

        return np.concatenate([F_p.flatten(), F_q.flatten()]) #
            concatenate in a unique vector to be solved by Newton

    p_guess, q_guess = explicita_Euler(p, q, h, masses, G) #
        explicit euler step used as initial guess
    x0 = np.concatenate([p_guess.flatten(), q_guess.flatten()])

    x_sol,_ = NEWTON(F, x0, TOL=1e-8, MAX_ITER=100)

    p_new = x_sol[:n*3].reshape(n,3)
    q_new = x_sol[n*3:].reshape(n,3)

    return p_new, q_new

def Symplectic_Euler(p, q, h, masses, G): # explicit in p, but
```

```
    implicit in q

    dp = Compute_Acc(q, masses, G)
    p_new = p + h*dp

    dq_new = compute_Vel(p_new, masses)
    q_new = q+h*dq_new

    return p_new, q_new

def stormer_verlet(p, q, h, masses, G):

    dp = Compute_Acc(q, masses, G) # half step in the momentum
    p_half = p+(h/2)*dp

    dq = compute_Vel(p_half, masses) # complete step in the
       position
    q_new = q + h*dq

    dp_new = Compute_Acc(q_new, masses, G) # 2nd half step in the
       momentum
    p_new = p_half + (h/2) * dp_new

    return p_new, q_new

def Simulate(method, h, T_final, p0,q0,masses, G):
    n_steps = int(T_final/h) + 1

    p = p0.copy()
    q = q0.copy()

    trajectories = np.zeros((n_steps, len(masses), 3))
    trajectories[0] = q

    for i in range(1,n_steps):
        p,q = method(p,q,h,masses,G)

        trajectories[i] = q

        if i% 1000 == 0 :
            print(f" steps {i}/{n_steps}") # print the progress in
                each 1000 iterations
    return trajectories

# def plot_trajectories(trajectories, title, ax): #### 2D plot

#     Planet_names = ['Sun', 'J', 'S', 'U', 'N', 'P']

#     for i in range(1,6): # 6 planets
```

```
#         ax.plot(trajectories[:,i,0], trajectories[:,i,1],
    linewidth = 0.5, alpha = 0.7)
#     for i in range(6):
#         ax.plot(trajectories[-1,i,0], trajectories[-1,i,1],'ko',
     markersize = 6)

#         ax.text(trajectories[-1,i,0], trajectories[-1,i,1], f'
    {Planet_names[i]}', fontsize = 10, fontweight = 'bold')

#     ax.set_xlabel('x [AU]')
#     ax.set_ylabel('y [AU]')
#     ax.set_title(title)
#     ax.axis('equal')
#     ax.grid(True, alpha = 0.3)

def plot_trajectories(trajectories, title, ax): # 3D plot
    fig = plt.figure(figsize=(10, 8))
    ax = fig.add_subplot(111, projection='3d')

    Planet_names = ['Sun', 'J', 'S', 'U', 'N', 'P']

    for i in range(1, 6):
        ax.plot(trajectories[:, i, 0],
                trajectories[:, i, 1],
                trajectories[:, i, 2],
                linewidth=0.5, alpha=0.7)

    for i in range(6):
        ax.scatter(trajectories[-1, i, 0],
                   trajectories[-1, i, 1],
                   trajectories[-1, i, 2],
                   color='black', s=50)
        ax.text(trajectories[-1, i, 0],
                trajectories[-1, i, 1],
                trajectories[-1, i, 2],
                f'  {Planet_names[i]}')

    ax.set_xlabel('x [AU]')
    ax.set_ylabel('y [AU]')
    ax.set_zlabel('z [AU]')
    ax.set_title(title)
    plt.show()

if __name__ == "__main__":

    T_final = 200000

    fig, axes = plt.subplots(2,2,figsize=(14,12))
    axes = axes.flatten()
```

```
print("\n1. Explicit Euler")

traj1 = Simulate(explicita_Euler, h = 10, T_final = T_final,
    p0=p_init, q0 = q_init, masses=masses, G=G)
plot_trajectories(traj1, 'Explicit Euler, h = 10', axes[0])

print("\n2. Implicit Euler")

traj2 = Simulate(implicit_Euler, h = 10, T_final = T_final, p0
    =p_init, q0 = q_init, masses=masses, G=G)
plot_trajectories(traj2, 'Implicit Euler, h = 10', axes[1])

print("\n3. Symplectic Euler")

traj3 = Simulate(Symplectic_Euler, h = 100, T_final = T_final,
     p0=p_init, q0 = q_init, masses=masses, G=G)
plot_trajectories(traj3, 'Symplectic Euler, h = 100', axes[2])

print("\n1. Stormer-Verlet")

traj4 = Simulate(stormer_verlet, h = 200, T_final = T_final,
    p0=p_init, q0 = q_init, masses=masses, G=G)
plot_trajectories(traj4, 'Stormer-Verlet, h = 200', axes[3])

plt.tight_layout()
```

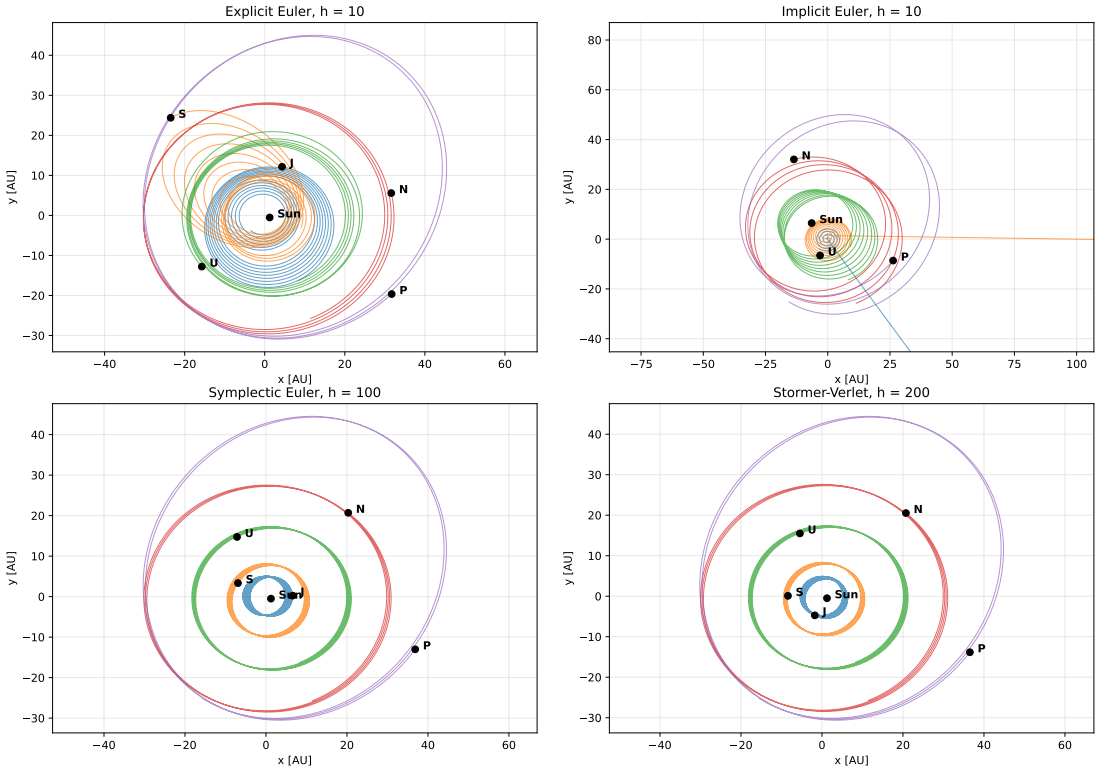



Figure 2.1: 2D view of the N-body problem plots (with zoom for Implicit Euler

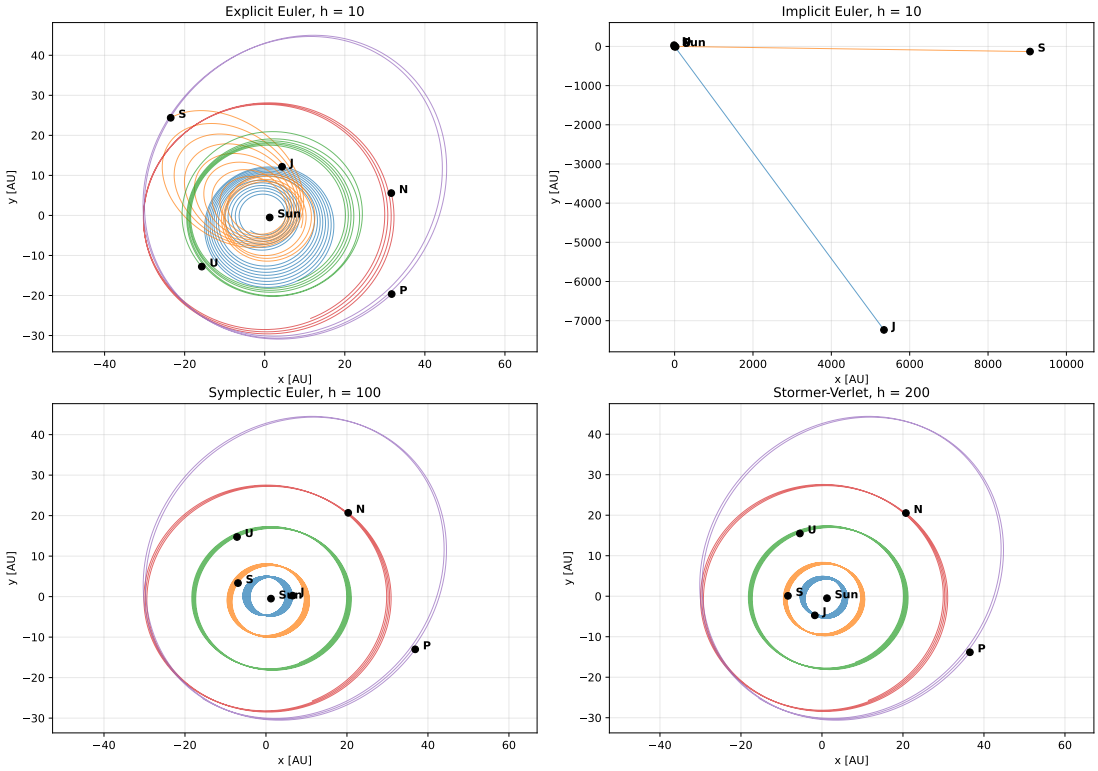
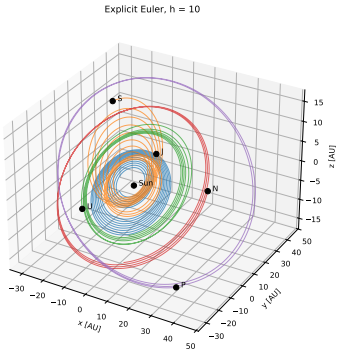
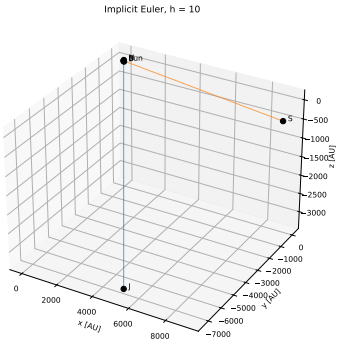


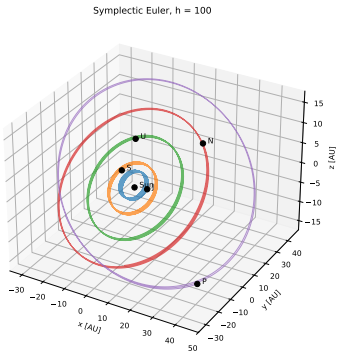

Figure 2.2: 2D view of the N-body problem plots



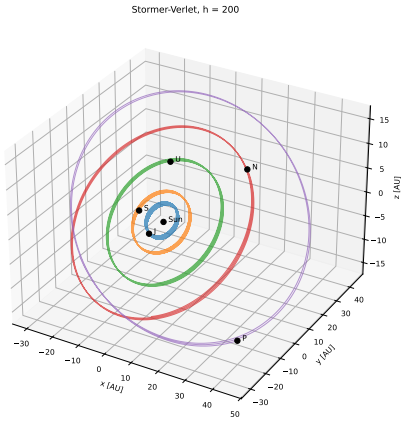

(a) 3D view of the solution using explicit Euler.




(b) 3D view of the solution using Implicit Euler.




(c) 3D view of the solution using Symplectic Euler.




(d) 3D view of the solution using Stormer-Verlet.

Figure 2.3: 3D view of the N-body problem

# 3 Task 2

The Yoshida method is a composition method that achieves fourth-order accuracy by combining three Störmer-Verlet steps with specially chosen weights. It is defined as,

$$\Psi_h = \phi_{\gamma_3 h} \circ \phi_{\gamma_2 h} \circ \phi_{\gamma_1 h} \tag{3.1}$$

where $\phi_h$ denotes a single Störmer-Verlet step and $\circ$ represents function composition. The coefficients are

$$\gamma_1 = \gamma_3 = \frac{1}{2 - 2^{1/3}} \approx 1.3512071919596576 \tag{3.2}$$

$$\gamma_2 = 1 - 2\gamma_1 \approx -1.7024143839193152 \tag{3.3}$$

The method proceeds by applying three consecutive Störmer-Verlet steps,

$$(\mathbf{p}_1, \mathbf{q}_1) = \phi_{\gamma_1 h}(\mathbf{p}_n, \mathbf{q}_n) \tag{3.4}$$

$$(\mathbf{p}_2, \mathbf{q}_2) = \phi_{\gamma_2 h}(\mathbf{p}_1, \mathbf{q}_1) \tag{3.5}$$

$$(\mathbf{p}_{n+1}, \mathbf{q}_{n+1}) = \phi_{\gamma_3 h}(\mathbf{p}_2, \mathbf{q}_2) \tag{3.6}$$

The properties of this method is

- Fourth-order accuracy: local error $\mathcal{O}(h^5)$, global error $\mathcal{O}(h^4)$
- Symplectic: preserves the Hamiltonian structure
- Time-reversible
- Allows much larger time steps than second-order methods while maintaining accuracy
- Note that $\gamma_2$ is negative, meaning the second step evolves "backwards" in time

The negative weight $\gamma_2$ is crucial for achieving fourth-order accuracy, as it allows the method to cancel third-order error terms through symmetric composition.
The discrete gradient method is an implicit integrator that exactly conserves the Hamiltonian $H(p,q)$ at each time step. For a Hamiltonian system written as,

$$\dot{\mathbf{x}} = J^{-1}\nabla H(\mathbf{x}), \quad \mathbf{x} = \begin{pmatrix} \mathbf{p} \\ \mathbf{q} \end{pmatrix}, \quad J = \begin{pmatrix} 0 & I \\ -I & 0 \end{pmatrix} \tag{3.7}$$

the method updates the state via,

$$\mathbf{x}_{n+1} = \mathbf{x}_n + h\, J^{-1}\bar{\nabla} H(\mathbf{x}_n, \mathbf{x}_{n+1}) \tag{3.8}$$

where $\bar{\nabla} H$ is the discrete gradient, chosen to satisfy,

$$H(\mathbf{x}_{n+1}) - H(\mathbf{x}_n) = \bar{\nabla} H(\mathbf{x}_n, \mathbf{x}_{n+1})^T \cdot (\mathbf{x}_{n+1} - \mathbf{x}_n) \tag{3.9}$$

For this implementation, we use the midpoint discrete gradient

$$\bar{\nabla} H(\mathbf{x}_n, \mathbf{x}_{n+1}) = \nabla H\left(\frac{\mathbf{x}_n + \mathbf{x}_{n+1}}{2}\right) \tag{3.10}$$

expanding in terms of momenta and positions, the update equations become,

$$\mathbf{p}_{n+1} = \mathbf{p}_n + h\left(-\frac{\partial H}{\partial \mathbf{q}}\Big|_{\text{mid}}\right) \tag{3.11}$$

$$\mathbf{q}_{n+1} = \mathbf{q}_n + h\left(\frac{\partial H}{\partial \mathbf{p}}\Big|_{\text{mid}}\right) \tag{3.12}$$

where the subscript "mid" indicates evaluation at the midpoint,

$$\mathbf{p}_{\text{mid}} = \frac{\mathbf{p}_n + \mathbf{p}_{n+1}}{2} \tag{3.13}$$

$$\mathbf{q}_{\text{mid}} = \frac{\mathbf{q}_n + \mathbf{q}_{n+1}}{2} \tag{3.14}$$

Since $(\mathbf{p}_{n+1}, \mathbf{q}_{n+1})$ appear implicitly on both sides, this forms a system of nonlinear equations,

$$F_{\mathbf{p}}(\mathbf{p}_{n+1}, \mathbf{q}_{n+1}) = \mathbf{p}_{n+1} - \mathbf{p}_n - h\left(-\frac{\partial H}{\partial \mathbf{q}}\Big|_{\text{mid}}\right) = \mathbf{0} \tag{3.15}$$

$$F_{\mathbf{q}}(\mathbf{p}_{n+1}, \mathbf{q}_{n+1}) = \mathbf{q}_{n+1} - \mathbf{q}_n - h\left(\frac{\partial H}{\partial \mathbf{p}}\Big|_{\text{mid}}\right) = \mathbf{0} \tag{3.16}$$

which we solve using Newton's method with the Störmer-Verlet solution as the initial guess. The properties of the discrete gradient method is

- Exact energy conservation: $H(\mathbf{p}_{n+1}, \mathbf{q}_{n+1}) = H(\mathbf{p}_n, \mathbf{q}_n)$ up to solver tolerance
- First-order accuracy: global error $\mathcal{O}(h)$
- Implicit method: requires solving nonlinear equations at each step
- Computationally expensive but provides superior long-term energy conservation
- Not symplectic in general, but preserves the Hamiltonian exactly

For task 2, was added the following lines to the code used in TASK 1

Listing 3: Main code to generate the plots for TASK 2

```
########################################## TASK 2
    ##############################################

def yoshida_step(p,q,h,masses, G):

    gamma_1 = 1.0/(2.0 - 2.0**(1.0/3.0))
    gamma_2 = 1.0 - 2.0 * gamma_1
    gamma_3 = gamma_1

    p, q = stormer_verlet(p, q, h*gamma_1, masses, G) # first step
    p, q = stormer_verlet(p, q, h*gamma_2, masses, G) # second
        step
    p, q = stormer_verlet(p, q, h*gamma_3, masses, G) # third step
    return p,q

def discrete_gradient(p,q,h,masses, G):
    n = len(masses)

    def F(x_flat):
        p_new = x_flat[:n*3].reshape(n,3)
        q_new = x_flat[n*3:].reshape(n,3)

        p_mid = (p + p_new) / 2 # mid point
        q_mid = (q + q_new) / 2

        dp_mid = Compute_Acc(q_mid, masses, G) # discrete gradient
            at mid point
```

```
        dq_mid = compute_Vel(p_mid, masses)

        F_p = p_new - p - h * dp_mid
        F_q = q_new - q - h * dq_mid

        return np.concatenate([F_p.flatten(), F_q.flatten()])

    p_guess, q_guess = stormer_verlet(p, q, h, masses, G)
    x0 = np.concatenate([p_guess.flatten(), q_guess.flatten()])

    x_sol, _ = NEWTON(F, x0, TOL = 1e-8, MAX_ITER=100)

    p_new = x_sol[:n*3].reshape(n,3)
    q_new = x_sol[n*3:].reshape(n,3)

    return p_new, q_new

    ################ TASK 2 ####################

    fig2, axes2 = plt.subplots(1,2,figsize=(14,12))
    axes2 = axes2.flatten()

    print("\n1. Yoshida Method")

    traj5 = Simulate(yoshida_step, h = 100, T_final = T_final, p0=
       p_init, q0 = q_init, masses=masses, G=G)
    plot_trajectories(traj5, 'Yoshida Method, h = 100', axes2[0])

    print("\n2. IDIscrete Gradient Method")

    traj6 = Simulate(discrete_gradient, h = 100, T_final = T_final
       , p0=p_init, q0 = q_init, masses=masses, G=G)
    plot_trajectories(traj6, 'Discrete Gradient, h = 100', axes2
       [1])


    plt.tight_layout()
```

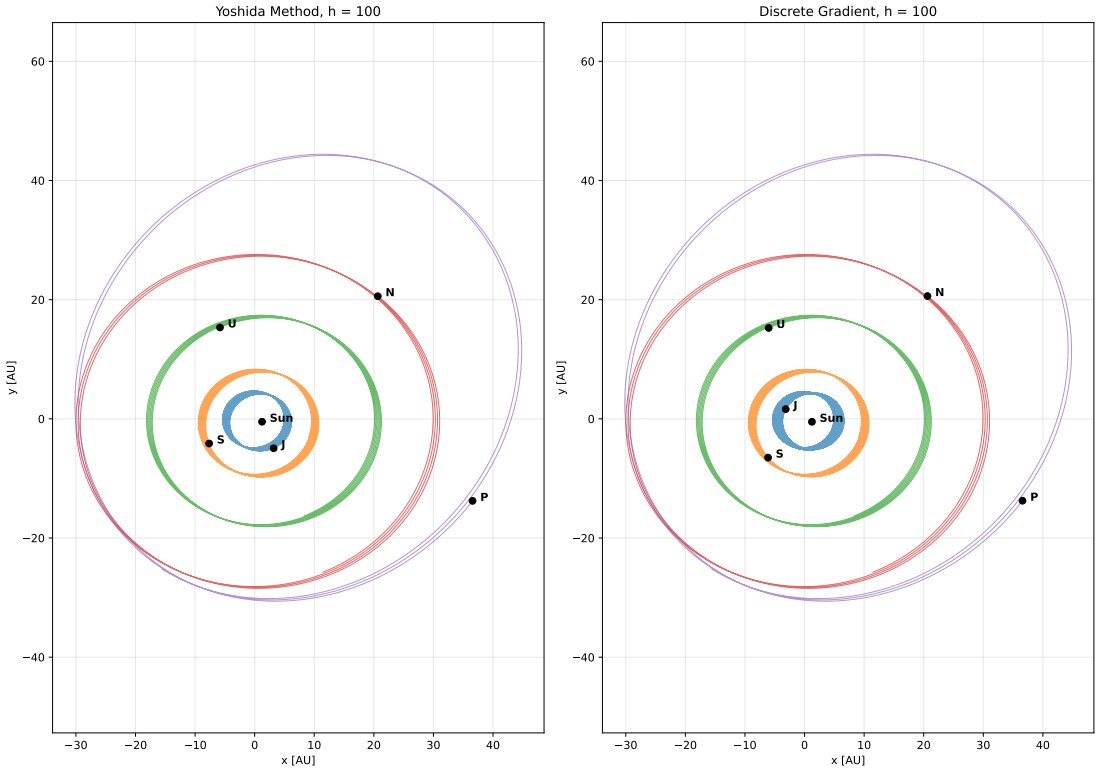
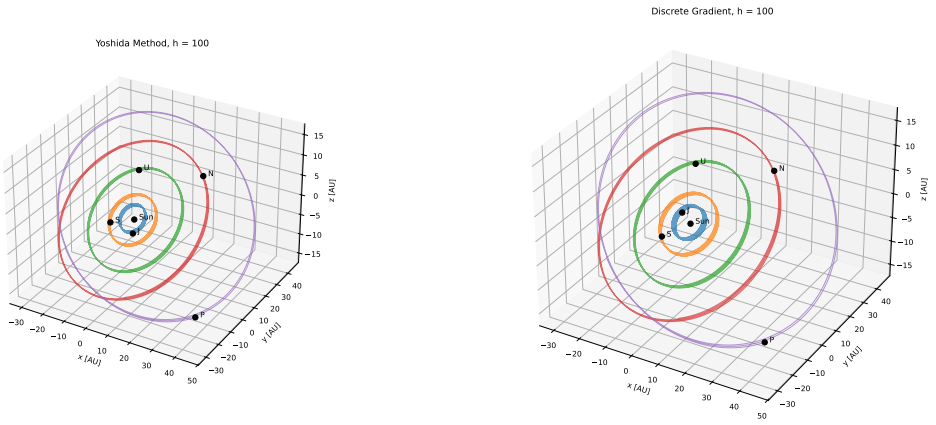



Figure 3.1: 2D view of the N-body problem using higher-order integrators




(a) 3D view of the solution using Yoshida methods.

(b) 3D view of the solution using Discrete gradient.

Figure 3.2: 3D view of the N-body problem using higher-order integrators

# 4 TASK 3

For TASK 3, a reference solution $\mathbf{q}_{\text{ref}}$ was computed using a high-order method (DOP853, order 8 R-K method, developed by Dormand & Prince (1980)) with adaptive step size, which was used to compare the solutions obtained by the methods in TASK 1 and TASK 2.

Listing 4: Code added for error and order analysis

```
###################### TASK 3
    ################################################

def Hamiltonian(p,q, masses, G):
    kinetic = 0.0
    potential = 0.0
    n = len(masses)
    for i in range(n):
        kinetic += np.sum(p[i]**2)/(2*masses[i]) # sum of kinetic
            energy
        for j in range(i+1,n): # loop over pairs, to void double
            count
            r = np.linalg.norm(q[i]-q[j]) # euclidian distance
                between bodies
            potential -= G*masses[i]*masses[j]/r

    return kinetic + potential # total hamiltoniam

def Simulate_final(method,h,T_final,p0,q0, masses, G):
    n_steps = int(T_final/h)
    p = p0.copy()
    q = q0.copy()

    for _ in range(n_steps):
        p, q = method(p,q,h,masses,G)
    return p,q

def Simulate_with_energy(method,h,T_final,p0,q0, masses, G):
    n_steps = int(T_final/h)+1 # total number of steps
    p,q = p0.copy(), q0.copy() # copy the initial state

    times = np.zeros(n_steps)
    energy = np.zeros(n_steps)

    for i in range(n_steps):
        times[i] = i*h
        energy[i] = Hamiltonian(p, q, masses, G) # energy at the
            currently state
        if i < n_steps -1:
            p, q = method(p,q,h,masses, G) # advances one step in
                time
    return times, energy
```

```
def energy_conservation_plot(methods, labels, h,T,p0,q0,masses,G):
    fig, axes = plt.subplots(2,3,figsize=(16,10))
    axes = axes.flatten()

    for idx, (method, label) in enumerate(zip(methods, labels)):
        print(f'\nSImulating {label} for energy...')
        times, energies = Simulate_with_energy(method, h, T, p0,
            q0, masses, G)

        H0 = energies[0]
        relative_error = (energies - H0)/abs(H0)

        axes[idx].plot(times, relative_error, linewidth=1)
        axes[idx].set_xlabel('Time [Days]')
        axes[idx].set_ylabel('Relative Energy Error')
        axes[idx].set_title(f'{label}, h={h}')
        axes[idx].grid(True, alpha=0.3)
        axes[idx].ticklabel_format(style='sci', axis='y',scilimits
            =(0,0))

    plt.tight_layout()
    return fig

def time_reversibility_test(methods, labels, h_values,n_steps, p0,
    q0, masses, G):

    def time_reversal(p,q):
        return -p, q

    print("\nReversibility time")
    print(f"Number of forward/backward steps: {n_steps}")

    fig, ax = plt.subplots(figsize=(10,7))
    for method, label in zip(methods, labels):
        print(f'\n{label}:')

        errors = []

        for h in h_values:
            # Initial state
            p, q = p0.copy(), q0.copy()

            # n forward steps
            for _ in range(n_steps):
                p, q = method(p, q, h, masses, G)

            # Temporal reversion
            p, q = time_reversal(p, q)
```

```
            # nn forward steps (= time traveling (Joke))
            for _ in range(n_steps):
                p, q = method(p, q, h, masses, G)

            # Time reversal
            p, q = time_reversal(p, q)

            error = np.linalg.norm(np.concatenate([p.flatten(), q.
                flatten()]) - np.concatenate([p0.flatten(), q0.
                flatten()]))

            errors.append(error)
            print(f" H = {h:6.1f}: error = {error:.3e}")

        ax.loglog(h_values, errors, 'o-', label=label, linewidth
            =2,markersize=6)

    ax.axhline(y=np.finfo(float).eps, color='red', linestyle='--',
        linewidth=2, label=f'Machine Precision')

    ax.set_xlabel('Step size h')
    ax.set_ylabel('Reversibility Erros')
    ax.set_title(f'Time Reversibility Test (n={n_steps} steps)',
        fontsize=14)
    ax.grid(True, which='both', alpha = 0.3)
    ax.legend()
    plt.tight_layout()

    return fig

def reference_solutions(T, p0, q0):
    n = len(masses)
    y0 = np.concatenate([p0.flatten(), q0.flatten()])

    def system_ode(t, y):
        p = y[:n*3].reshape(n, 3)
        q = y[n*3:].reshape(n, 3)

        dp = Compute_Acc(q, masses, G)
        dq = compute_Vel(p, masses)

        return np.concatenate([dp.flatten(), dq.flatten()])

    sol = solve_ivp(system_ode, (0, T), y0,
                    method='DOP853',
                    rtol=1e-12, atol=1e-14)

    q_ref = sol.y[n*3:, -1].reshape(n, 3)
```

```
    return q_ref

# def reference_solutions(T,p0,q0):
#     h_ref = 2**(-7) # small tep for the reference solution
#     _, q_ref = Simulate_final(yoshida_step, h_ref, T, p0, q0,
   masses, G) # use the higher order method to the reference
   solution
#     return q_ref # return the final position

def global_position_error(method,h,T,p0,q0,q_ref):
    _ , q_num = Simulate_final(method, h, T, p0, q0,masses, G)
    return np.linalg.norm(q_num - q_ref) # global error

def order_plot_all(methods, labels, k_max, T, p0, q0):
    q_ref = reference_solutions(T, p0, q0) # calculate the
       reference solution
    plt.figure()

    total_sims = len(methods)*k_max
    sim_count = 0

    for method, label in zip(methods, labels): # loop over all
       methods
        hs = [] # step list
        errors = [] # errors list

        print(f"\n{'='*50}")
        print(f"Method: {label}")
        print(f"{'='*50}")

        for k in range(1,k_max+1):
            sim_count += 1
            h = 2**(-k)
            n_steps = int(T/h)
            print(f"  [{sim_count}/{total_sims}] k={k}, h={h:.6f},
                steps={n_steps}... ", end='', flush=True)

            err = global_position_error(method, h, T, p0, q0,
               q_ref) # global error
            hs.append(h)
            errors.append(err)

            print(f"* error={err:.3e}")

        plt.loglog(hs, errors, 'o-', label = label)

    h_ref = np.array([hs[0],hs[-1]])

    plt.loglog(h_ref, h_ref*errors[0]/hs[0], '--', color='gray',
```

```
    alpha=0.5, label="Order 1")
plt.loglog(h_ref, (h_ref**2)*errors[0]/(hs[0]**2), '--', color
    ='gray', alpha=0.5, label="Order 2")
plt.loglog(h_ref, (h_ref**4)*errors[0]/(hs[0]**4), '--', color
    ='gray', alpha=0.5, label="Order 4")

plt.xlabel('h',fontsize=12)
plt.ylabel('Global Error',fontsize=12)
plt.title('Convergence Order plot', fontsize = 14)
plt.grid(True, which='both',alpha=0.3)
plt.legend()

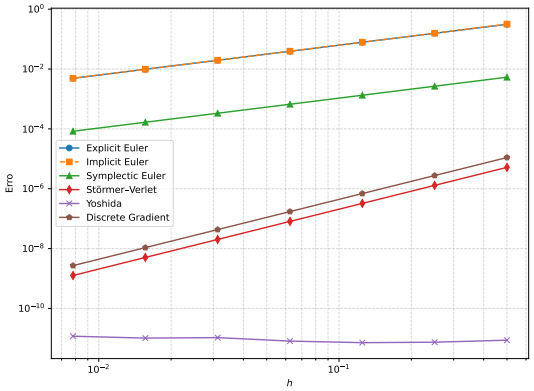

##################### TASK 3
    ################################################

methods = [
    explicita_Euler,
    implicit_Euler,
    Symplectic_Euler,
    stormer_verlet,
    yoshida_step,
    discrete_gradient
    ]
labels = [
    'explicite_Euler',
    'implicit_Euler',
    'Symplectic_Euler',
    'stormer_verlet',
    'yoshida_step',
    'discrete_gradient'
    ]

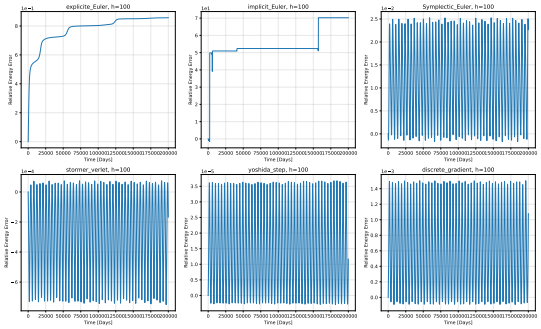
print("\n1. Order plot")
order_plot_all(methods, labels, k_max=7,T=5000, p0=p_init, q0=
    q_init)
plt.tight_layout()

print("\n2. Energy conservation")
fig_energy = energy_conservation_plot(methods, labels, h=100,
    T=200000, p0=p_init, q0=q_init, masses=masses, G=G)

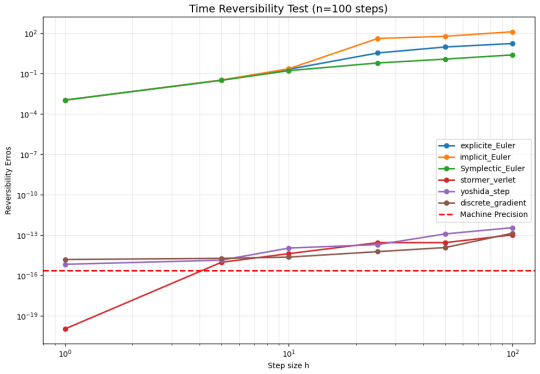
print("\n3. Time reversibility")
h_values = [100,50,25,10,5,1]
n_steps = 100
fig_rev = time_reversibility_test(methods, labels, h_values,
    n_steps, p0=p_init, q0=q_init, masses=masses, G=G)
```




(a) Order P graph from all methods used




(b) Energy conservation from each method




(c) Time reversibility from each method

Figure 4.1: Plots from task 3

# 5 Conclusion

Overall, the results clearly show the differences in stability, accuracy, and computational cost among the integrators. Explicit Euler and Implicit Euler performed poorly: Explicit Euler was unstable and caused Jupiter and Saturn to crash into the Sun, while Implicit Euler took a very long time to run and ended up throwing planets far away, confirming what is mentioned in the book. On the other hand, the symplectic methods, particularly Symplectic Euler and Stormer-Verlet, delivered very good results with fast runtimes. Stormer-Verlet showed very small oscillatory energy errors, making it the gold standard for long-term orbital simulations, while Symplectic Euler remained stable with bounded oscillations, although less precise.
High-order methods like Yoshida and the Discrete Gradient performed excellently in conserving energy. The experimental order $p$ of the Yoshida method could not be accurately measured because the truncation error was already negligible and round-off error dominated. Time-reversibility tests confirmed the symplectic nature of Stormer-Verlet, Yoshida, and Discrete Gradient, showing errors around $10^{-13}$ to $10^{-20}$, which are essentially at machine precision. Comparing the methods by their final energy errors, Explicit Euler and Implicit Euler

clearly fail, while Symplectic Euler is good, Stormer-Verlet and Yoshida are excellent, and Discrete Gradient is very solid, combining small errors with bounded oscillations.

Listing 5: Code use to calculade order-p

```
import matplotlib.pyplot as plt
import numpy as np

# steps
h = np.array([0.5, 0.25, 0.125, 0.0625, 0.03125, 0.015625,
    0.0078125])

# errors
explicit_Euler = np.array([3.116e-01, 1.565e-01, 7.843e-02, 3.926e
    -02,
                           1.964e-02, 9.822e-03, 4.912e-03])

implicit_Euler = np.array([3.172e-01, 1.579e-01, 7.878e-02, 3.934e
    -02,
                           1.966e-02, 9.828e-03, 4.913e-03])

symplectic_Euler = np.array([5.329e-03, 2.665e-03, 1.333e-03,
    6.665e-04,
                             3.332e-04, 1.666e-04, 8.331e-05])

stormer_verlet = np.array([5.178e-06, 1.295e-06, 3.236e-07, 8.091e
    -08,
                           2.023e-08, 5.057e-09, 1.262e-09])

yoshida_step = np.array([8.692e-12, 7.452e-12, 7.185e-12, 8.105e
    -12,
                         1.061e-11, 1.021e-11, 1.181e-11])

discrete_gradient = np.array([1.104e-05, 2.760e-06, 6.900e-07,
    1.725e-07,
                              4.312e-08, 1.078e-08, 2.696e-09])

print(h)
print(explicit_Euler)
print(len(h), len(explicit_Euler))


# plot log-log
plt.figure(figsize=(8,6))

plt.loglog(h, explicit_Euler, 'o-', label='Explicit Euler')
plt.loglog(h, implicit_Euler, 's--', label='Implicit Euler')
plt.loglog(h, symplectic_Euler, '^-', label='Symplectic Euler')
plt.loglog(h, stormer_verlet 'd-', label='Stormer-verlet')
plt.loglog(h, yoshida_step, 'x-', label='Yoshida')
```

```
plt.loglog(h, discrete_gradient, 'p-', label='Discrete Gradient')

plt.xlabel(r'$h$')
plt.ylabel('Erro')
plt.legend()
plt.grid(True, which="both", ls="--", alpha=0.5)
plt.tight_layout()


def order_loglog(h, E):
    logh = np.log10(h)
    logE = np.log10(E)
    # polyfit return [slope, intercept]
    p, logC = np.polyfit(logh, logE, 1)
    return p

#order_p calculus
p_explicit = order_loglog(h, explicit_Euler)
p_implicit = order_loglog(h, implicit_Euler)
p_symplectic = order_loglog(h, symplectic_Euler)
p_stormer = order_loglog(h, stormer_verlet)
p_yoshida = order_loglog(h, yoshida_step)
p_discrete = order_loglog(h, discrete_gradient)

print("Ordem experimental p:")
print(f"Explicit Euler: {p_explicit:.3f}")
print(f"Implicit Euler: {p_implicit:.3f}")
print(f"Symplectic Euler: {p_symplectic:.3f}")
print(f"stormer-verlet: {p_stormer:.3f}")
print(f"Yoshida: {p_yoshida:.3f}")
print(f"Discrete Gradient: {p_discrete:.3f}")
```

# 6 STEPS TO BE DONE

- **TASK 1:**
  - Implement the numerical integration scheme used in the respective section. For implicit methods, make use of your implementation of Newtons method to solve the arising non-linear system numerically,
  - Perform the numerical simulation and create the plots you aim to reproduce
- **TASK 2:**
  - Implement the numerical integrators: Yoshida: $\Psi_h = \phi_{\gamma_3 h} \circ \phi_{\gamma_2 h} \circ \phi_{\gamma_1 h}$ with $\gamma_1 = \gamma_3 = \frac{1}{2-2^{1/3}}$, $\gamma_2 = 1 - 2\gamma_1$, and the $\phi_h$ denoting the Verlet/Leapfrog method
  - The discrete gradient method $x_n + 1 = x_n + hJ^{-1}\bar{\nabla} H(x_n, x_{n+1}$ for the system $x' = J^{-1}\nabla H(x)$ where $\bar{\nabla H}$ denotes the midpoint discrete gradient.

    Then perform numerical simulations for these two methods and create plots similar to those provided in the textbook.
- **TASK 3:**
  - for both the numerical simulation schemes used in the textbook and the schemes from task 2, perform numerical simulations to create the following plots:
    * order plot: global error vs step size for different step sizes $h = 2^- k,\ k = 1, 2, ...k_m$, create the plot with log-log scale using plt.loglog(). The slope of the line should be $p$ for order-p method
    * plot the data points $(tn, H(x_n))$ to check in which sense the hamiltonian is preserval or not.
    * check the violation in the time-reversibility by computing the difference $\rho \circ \phi_h \circ \rho\phi_h(y_0) - y_0$

      for large step size $h$. Is the error within machine precision?
- **TASK 4:**
  - Write a report in the style of the textbook (i.e. LaTex or Jupiter notebook) and write down all your results + discussion/observations of the results